Introduction to React

- Why React?
- What Is React?
- Virtual DOM: The Power of React
- Understanding JSX Syntax
- Rendering JSX Elements in React

React Components

- Introduction to React Components
- Passing Props to Components
- Styling Components in React
- Handling Events in React Components
- Creating Stateless Components

Hooks: Managing States and Effects in React Components

- Understanding Hooks in React
- The useState Hook
- The useEffect Hook
- The useContext Hook
- The useRef Hook
- The useMemo Hook

React Router and Navigations

- Introduction to React Router
- Defining Routes in React
- Navigating Between Pages in React
- Dynamic Routing
- Nested Routes

Introduction to React

Why React?

Explore the common pitfalls of creating dynamic user interfaces with plain JavaScript and understand how React addresses these challenges through declarative syntax, virtual DOM updates, component encapsulation, and simplified event handling.

Click on this to download for Displaying a welcome message and an alert using JavaScript

The code works by manually injecting HTML into the DOM using innerHTML and attaching an event listener to the button. While it achieves the goal, it highlights some challenges of building dynamic UIs with plain JavaScript. As the application becomes more complex,

we would need to handle frequent DOM updates: Each time the state of the application changes, the developer must manually locate and update specific DOM elements. For example, dynamically adding or removing items in a list requires carefully managing innerHTML or creating and appending new DOM elements.

Re-render multiple elements: The renderUI function replaces the entire content of the container element using innerHTML every time it is called. If new functionality requires only partial updates (e.g., updating just the heading or button text), the entire UI is unnecessarily re-rendered, leading to inefficiency.

Ensure event listeners are correctly managed: Every time renderUI is called, a new event listener is added to the button, leading to a potential buildup of duplicate listeners. This not only wastes memory but can also cause bugs, such as the same event being triggered multiple times.

This approach is not scalable for larger applications: If new requirements are added, such as multiple buttons, dynamic lists, or more complex interactions, the developer would need to manually manage each element, its content, and its event listeners. This quickly becomes unmanageable as the number of elements grows.

How React addresses these challenges

React's declarative and component-based approach offers solutions to the challenges faced when using plain JavaScript for dynamic UIs. Let's reimplement the above scenario and examine how React addresses each specific challenge present in the plain JavaScript implementation.

Click on this to download for Displaying a welcome message and an alert using React

Let's break down the above React code to explain the solutions that React provides.

Efficient DOM updates with the virtual DOM

In plain JavaScript, frequent updates to the DOM can lead to inefficiencies. For example, updating the heading text or adding a button dynamically requires directly modifying the DOM using methods like innerHTML. On the other hand, in React:

```
<h1>Welcome to React</h1>
<button onClick={handleClick}>Click Me</button>
```

These lines describe *what* the UI should look like, rather than *how* to update it. React's Virtual DOM efficiently calculates the difference between the current and new UI states, ensuring minimal and optimized updates to the real DOM.

Declarative syntax for defining the UI

In plain JavaScript, we need to imperatively define every step to create and modify DOM elements. This can result in repetitive and error-prone code. However, in React:

```
return (
  <div>
    <h1>Welcome to React</h1>
    <button onClick={handleClick}>Click Me</button>
  </div>
);
```

The return statement declaratively defines the structure of the UI. We simply describe the desired outcome, and React takes care of rendering it. There's no need to manually manipulate the DOM with methods like document.createElement or innerHTML.

Encapsulation with components

In plain JavaScript, rendering the UI and attaching event listeners are typically combined in a single function, leading to tightly coupled and less reusable code. But in React:

```
const App = () => { ... };
```

The function (e.g., App) is a **component**, a reusable, self-contained piece of UI. Components encapsulate both the UI definition and its associated behavior (like event handling), making the code modular and maintainable.

Simplified event handling

In plain JavaScript, we must manually find the DOM element and attach event listeners using the addEventListener method. Managing event listeners for dynamic elements is error-prone and can lead to duplicate listeners or memory leaks. On the other side, in React:

```
<button onClick={handleClick}>Click Me</button>
```

The onClick attribute directly binds the handleClick function to the button's click event. React ensures that event listeners are efficiently managed and cleaned up when the component is removed, preventing issues like memory leaks or duplicated handlers.

Plain JavaScript vs. React

Comparison of the provided JavaScript and React implementations for rendering a heading and a button that shows an alert on click.

Aspect	Plain JavaScript	React
Rendering the UI	The `renderUI` function uses `innerHTML` to inject HTML into the DOM manually.	The `App` component declaratively defines the UI structure using JSX.
Updating the UI	Each time the UI needs to change, the entire container (`<div>`) is re-rendered, replacing all its content.	React's virtual DOM ensures that only the part of the UI that changes (e.g., `<h1>`) is updated.
Event handling	A new event listener (`addEventListener`) is attached to the button every time `renderUI` function is called, risking duplicate listeners.	The `onClick` handler is defined inline within the JSX, automatically managed by React.
Modularity	Rendering logic and event handling are tightly coupled in the same function, reducing reusability.	The `App` component encapsulates both UI and behavior, making it reusable and easier to maintain.
State management	No explicit state management. UI updates must be synchronized manually, increasing complexity.	State (e.g., `useState`) drives the UI automatically. Updates to state trigger a re-render only where needed.

By addressing the challenges of plain JavaScript, some of which we saw above, React offers an efficient way to build dynamic, interactive, and scalable user interfaces.

What Is React?

React is a popular JavaScript library for building user interfaces. It's also especially useful for single-page applications (SPAs). It was created and is maintained by Facebook (now Meta), and has grown into one of the most widely used libraries for front-end development. At its core, React helps developers create dynamic and interactive user interfaces with less effort and better performance.

Unlike traditional methods of updating the user interface directly via the DOM, React adopts a declarative approach. Developers describe what the UI should look like at any given state, and React takes care of updating the actual DOM to match. This eliminates the need for manually writing complicated DOM manipulation logic, making code easier to write and maintain.

React ecosystem

The React ecosystem includes tools and libraries that complement its functionality. Let’s explore some key members of this ecosystem:

- **React Router:** It enables navigation between pages in a React application. For example, in a blogging platform, React Router can enable navigation between pages like the home page, individual blog posts, and an about page.
- **Redux:** It helps manage complex application state. Imagine a shopping cart application where we need to track items added to the cart, user details, and payment status across various components. Redux ensures all components have access to the required data without creating a tangled web of dependencies.
- **Next.js:** A framework built on React for server-side rendering and static site generation. For instance, an e-commerce website using Next.js can serve pre-rendered product pages to users, reducing load times and improving search engine rankings.
- **React Native:** It extends React's capabilities to mobile app development. For example, a travel booking app built with React Native can provide a seamless user experience on both iOS and Android without requiring separate codebases for each platform.
- **MERN stack:** The MERN stack integrates React with JavaScript-based technologies and MongoDB to enable full-stack development. For example, a task management app built with the MERN stack can use React for the user interface, Express.js for backend logic, Node.js to run the server, and MongoDB to store tasks and user data.

This ecosystem has made React the go-to choice for modern web development, from creating simple websites to building complex applications used by millions of users.

React Architecture:

React architecture explains how a React application is structured and how data flows inside it.

At a high level, React follows:

- Declarative UI updates
- Component-based architecture
- Virtual DOM rendering
- Unidirectional data flow

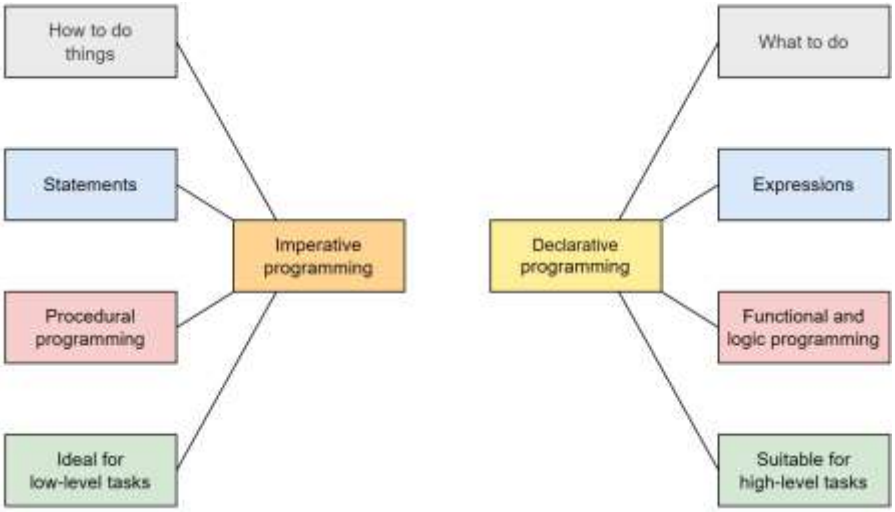
👉 React focuses mainly on the View layer of an application.

Why React is popular

React has become immensely popular for several reasons. Let’s explore two of its standout features: the declarative UI approach and its component-based architecture.

Declarative UI: Simplifying user interface updates

Traditional web development often involves imperative programming, where developers explicitly define every step needed to update the DOM. This can quickly become error-prone and difficult to manage as the application grows.



Key differences between imperative and declarative programming

React, on the other hand, adopts a declarative approach. We describe the desired UI for a specific state, and React efficiently updates the DOM to match it. For example, if a user interacts with a button that changes some data, React ensures the UI reflects the updated state without us having to write tedious DOM manipulation code.

Component-based architecture: Building reusable, modular code

In React, everything revolves around components. **Components** are the building blocks of React applications. They are self-contained units of code that define a piece of the UI and its behavior. For example, a button, a navigation bar, or a user profile card can each be a component.

Components promote reusability and modularity:

- **Reusability:** Components can be reused across different parts of an application, saving time and reducing code duplication.
- **Modularity:** Applications are divided into smaller, manageable pieces, making it easier to develop, debug, and maintain.

Let's look at this way: Imagine we're assembling a car. Instead of building it from scratch every time, we use pre-made parts like the engine, wheels, and seats. Similarly, React allows us to "assemble" our application using components.

Example: A simple React component

An example component in React:

```
function Welcome() {
  return <h1>Welcome to React!</h1>;
}
```

When this component is rendered in a React application, it displays the Welcome to React! heading.

React’s declarative approach and component-based architecture simplify UI development a lot. It's easier to create dynamic and interactive applications while keeping the codebase organized and maintainable, and React helps us do that.

Virtual DOM: The Power of React

The **real DOM** (browser DOM) is the actual DOM tree that the browser renders and displays to the user. This means any changes made to the real DOM are immediately reflected on the user's screen, but direct manipulation can be inefficient for complex or frequent updates due to the performance cost of re-rendering the UI. In contrast, the **virtual DOM** is a lightweight, in-memory representation of the real DOM used by libraries like React to optimize updates. By handling updates in this way, React minimizes the performance impact on the browser and ensures a smoother user experience.

Real vs. virtual DOM

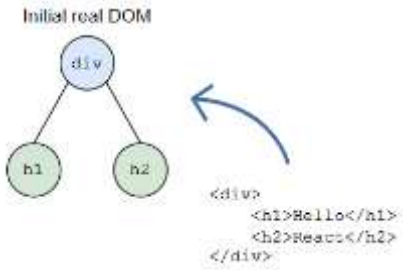
Understanding the difference between the real DOM and the virtual DOM is essential to grasp how React optimizes UI updates. In the following example, we’ll explore how React efficiently updates the UI by using the virtual DOM to minimize changes to the real DOM.

Step-by-step explanation

We have a simple React application that initially renders two header elements displaying Hello and React. We want to update the UI by adding a third header element displaying Element. The goal is to understand how React handles this update efficiently using the virtual DOM.

Step 1: Initial real DOM

Examine the starting point of our application by viewing the initial HTML structure rendered in the browser. In this step, the initial real DOM rendered by the browser is created.



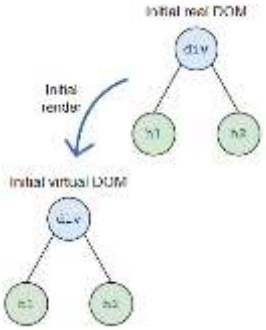
Step 1: Browser builds the initial real DOM tree

The initial real DOM consists of a element containing two header elements: <h1>Hello</h1> and <h2>React</h2>. This structure displays the texts Hello and React to the user. It’s the actual DOM rendered by the browser and displayed on the screen.

Step 2: Initial rendering and virtual DOM creation

In this step, React creates a virtual DOM that mirrors the initial real DOM. Let's see how React represents the initial UI in memory using the virtual DOM:

```
const virtualDOM = {
   type: 'div',
   props: {},
   children: [
      {
         type: 'h1',
         props: {},
         children: ['Hello'],
      },
      {
         type: 'h2',
         props: {},
         children: ['React'],
      },
   ],
};
```



Step 2: Build the initial virtual DOM by mirroring the current state of the real DOM

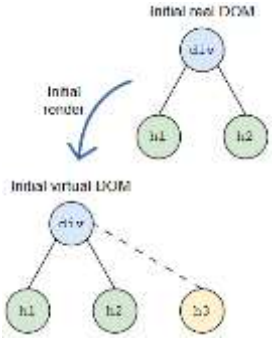
Note: Don't worry if some of the terms in this code, like props, or children, are unfamiliar to you. We'll cover these concepts in detail in later lessons. For now, focus on understanding the bigger picture.

This virtual DOM is a lightweight, in-memory representation of the UI, allowing React to manage changes efficiently.

Step 3: UI update triggered

Next, update the UI by adding a new header element:

<h3>Element</h3>

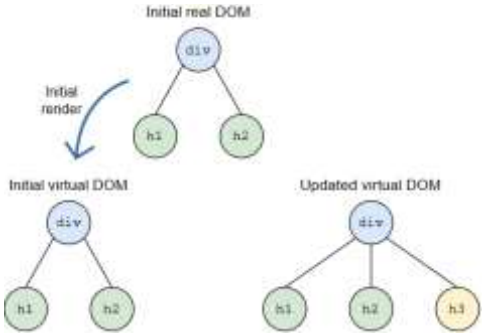


Step 3: A new element (<h3>Element</h3>) is to be added

Step 4: Updated virtual DOM creation

In this step, React creates an updated virtual DOM that includes the new header element. Let's see how React represents the updated UI in the virtual DOM after the planned change:

```
const updatedVirtualDOM = {
   type: 'div',
   props: {},
   children: [
      {
         type: 'h1',
         props: {},
         children: ['Hello'],
      },
      {
         type: 'h2',
         props: {},
         children: ['React'],
      },
      {
         type: 'h3',
         props: {},
         children: ['Element'],
      },
   ],
};
```



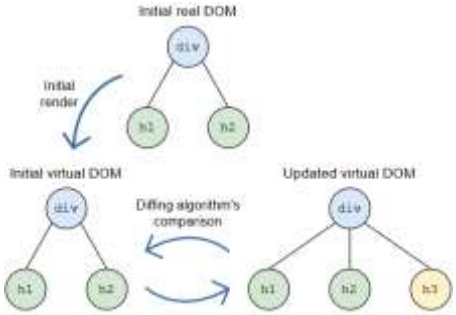
Step 4: React generates an updated virtual DOM reflecting the desired UI

The updated virtual DOM now includes the new <h3> element, reflecting the desired UI after the update.

Step 5: Diffing the virtual DOMs

React now compares the initial and updated virtual DOMs to identify what has changed. React uses it's diffing algorithm to determine the differences between the two virtual DOMs by:

- Comparing each node in the initial and updated virtual DOMs.
- Identify any additions, deletions, or modifications.

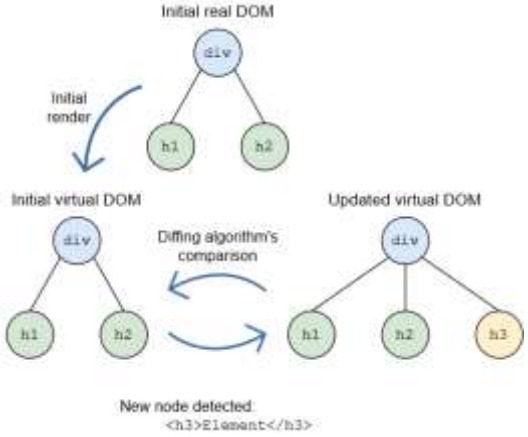


Step 5: React compares the initial and updated virtual DOMs to find differences

Step 6: Identifying changes

After diffing, React determines the specific changes needed. It reviews the findings of the diffing process to see what updates are necessary.

- **Unchanged nodes:**
  - <h1>Hello</h1>: No changes.
  - <h2>React</h2>: No changes.
- **New node detected:**
  - <h3>Element</h3>: This is a new node.



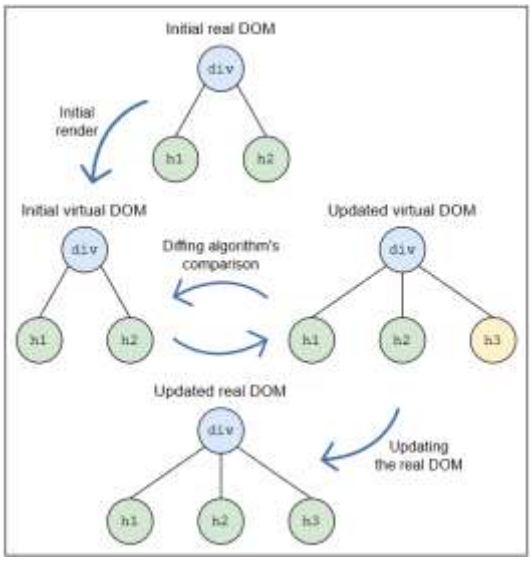
Step 6: The algorithm detects the new <h3> element as the only change

Step 7: Updating the real DOM

React updates the real DOM by applying changes identified during the diffing process, inserting the new element (<h3>Element</h3>) into the real DOM under the existing element. Finally, the browser renders the updated real DOM, displaying the new content to the user.

```
<div>
  <h1>Hello</h1>
  <h2>React</h2>
  <h3>Element</h3>
</div>
```

The updated real DOM with the new <h3> element



Step 7: React updates the real DOM by inserting the new element

This approach optimizes performance by reducing unnecessary DOM manipulations and ensures a smooth user experience. This is one powerful benefit of React's rendering system and how the virtual DOM optimizes UI updates.

Understanding JSX Syntax

When building user interfaces, developers often need to describe the structure of the UI while embedding logic for interactivity. Traditional JavaScript requires manipulating the DOM directly, which can be verbose and challenging to maintain as applications grow.

JSX (JavaScript XML)

React solves this problem with **JSX**, a syntax that lets us describe the structure of our UI in a concise, HTML-like format while seamlessly integrating JavaScript logic. It's an optional syntax extension for JavaScript that looks like HTML but is compiled into JavaScript. It allows us to write UI code in a way that's intuitive and closer to how our UI will look in the browser.

A JSX element defining an <h1> tag with the Hello, JSX! text:

const element = <h1>Hello, JSX!</h1>;

React compiles this JSX into JavaScript. The compiled version of the above code would look like this:

const element = React.createElement('h1', null, 'Hello, JSX!');

JSX syntax rules

To write JSX effectively, we need to follow certain rules. These rules ensure that JSX is syntactically correct and works seamlessly with JavaScript.

Rule 1: JSX must have one parent element

JSX expressions must be wrapped in a single parent element. If we want to render multiple elements, wrap them inside a . However, this can lead to unnecessary DOM elements, especially when the wrapper doesn't serve a meaningful purpose.

Click here to download the code for JSX elements wrapped inside a single valid syntax

React solves this problem with **React fragments** <>...</>, which allow us to group multiple JSX elements without adding an extra node to the DOM. Fragments are lightweight and invisible in the rendered output, making them a cleaner choice for grouping elements.

Click here to download the code for JSX elements wrapped inside fragments

Rule 2: Embedding JavaScript in JSX

We can embed JavaScript expressions within JSX using curly braces {}. This is useful for dynamically rendering data.

Click here to download the code for Embedding the name variable inside a JSX element

Rule 3: JSX attributes

JSX attributes work similarly to HTML attributes, but use **camelCase** for multi-word attributes like className instead of class.

const button = <button className="btn-primary">Click Me</button>;

A JSX element with the className attribute for styling

JSX and JavaScript

JSX supports all JavaScript capabilities, allowing us to include loops, conditional statements, and function calls.

Example: Conditionally render JSX with a ternary operator

Let's use a ternary operator in JSX to dynamically render different elements based on a condition.

Click here to download the code for a ternary operator to conditionally render JSX elements

- **Line 8:** Embeds a ternary expression inside JSX to decide which message to display based on the isLoggedIn value.

Example: Rendering lists in JSX

Let's use the map method in JSX to dynamically render a list of items as React elements.

Click to download the code for Rendering a list dynamically using the map function inside JSX

- **Line 5:** Declares an array items.
- **Line 8–10:** Maps over the array to render each item as a <li> element with a unique key.

Rendering JSX Elements in React

React doesn't directly render JSX elements to the browser. Instead, it relies on the react-dom package, which provides methods to interact with the browser's DOM. It serves as the bridge between React's virtual DOM and the actual browser DOM.

The createRoot API

In React 18 and later, react-dom handles rendering these components into the DOM and updating them efficiently using an API called createRoot. It connects the React application to the browser's DOM and is responsible for displaying our UI. This new API offers better performance and enables concurrent rendering, making React applications more efficient and responsive.

React introduced the createRoot API in React 18 as part of its updates to the React rendering engine. It replaces the older ReactDOM.render method to provide better performance and support for new features like **concurrent rendering**.

How createRoot works

The createRoot method is used to create a React root container for rendering. Once the root container is created, its render method is used to render React elements into the DOM.

The createRoot method takes one argument:

- **Where to render:** A DOM node in the HTML document where the React content will be displayed.

Once the root container is created, the render method takes:

- **What to render:** A React element or JSX expression that describes the UI.

Typically, React applications render their content inside a single HTML container, often an element with the ID root.

Rendering a simple JSX element

Let's render a simple JSX element to the browser using createRoot.

Click to download for Rendering an <h1> element with the text "Hello, React!" to the browser

- **Lines 1–2:** Import the React library and the createRoot method from react-dom/client.
- **Line 5:** Create a JSX element named element with the text Hello, React!.
- **Line 7:** Create a React root container using createRoot, targeting the DOM node with the ID root.
- **Lines 8–12:** Use the render method of the root container to render the element inside the <StrictMode> component, which activates additional checks and warnings to help identify potential issues during development.

Note: In React, the <StrictMode> tool helps us find common mistakes in our code during development. It doesn't show anything on the screen, but it quietly checks the components inside it and gives helpful warnings if something might cause problems in the future. It's like a helpful assistant that watches our code and gives suggestions behind the scenes—only while we're building the app, not when users are using it. For example, if we accidentally write code that runs twice when it shouldn't—like fetching data from a server every time the component updates—<StrictMode> will show a warning to help us fix it before it becomes a bigger issue.

Dynamic updates with createRoot

Even though React components are the preferred way to handle dynamic updates, we can manually call the render method multiple times to update content. Each call replaces the previous content in the target DOM node. Let's explore this with the code below:

Click here to download for Rendering and updating a counter dynamically every second

- **Line 5:** Declare a variable counter to track the count.
- **Lines 7–14:** Define the updateCounter function, which creates a JSX element displaying the current counter value and renders it inside the <StrictMode> component using the render method.
- **Line 16:** Create a React root container using createRoot.
- **Line 19:** Call updateCounter initially to render the counter.
- **Lines 22–25:** Use setInterval to increment counter and re-render the updated value every second.

Handling dynamic data

JSX allows embedding JavaScript expressions to render dynamic content. React dynamically updates the DOM when such expressions change.

Click here to download for Rendering user data dynamically with JSX

- **Lines 5–8:** Declares a user object with name and age properties.
- **Lines 10–15:** Embeds JavaScript expressions {user.name} and {user.age} within JSX to display dynamic data.
- **Lines 17–22:** Create a React root container using createRoot and render the JSX element inside <StrictMode> to enable additional development checks.

React Components

Introduction to React Components

Building complex user interfaces can be challenging, but React simplifies this process by breaking the UI into manageable, reusable pieces called components. Understanding and being able to apply components well is essential for any React developer.

Components are the fundamental units of a React application. Think of them as JavaScript functions or classes that accept inputs and return React elements describing what should appear on the screen.

Why components matter

- **Reusability:** Components allow us to reuse code, making our application more maintainable.

- **Modularity:** Breaking the UI into components makes it easier to manage and debug.
- **Readability:** A well-structured components hierarchy improves the readability of our code.

The Two Types of React Components

Components are relatively easy to explain. They allow us to de-construct complex user interfaces into multiple parts. Ideally, these parts are reusable, isolated, and self-contained. They can deal with any input from the outside and describe what will be rendered on the screen by whatever they define in the render() method.

We can cluster components into two different versions:

1. **Function components**, which are expressed in the form of a function.
2. **Class components**, which build upon the newly introduced ES2015 classes.

Until recently, the term **stateless functional component** was prevalent because the state could only be managed in class components before the introduction of React Hooks. From React 16.8.0 onwards, the state is managed using Hooks.

Function components

The simplest way to define a React component is through a function component. You might have guessed it, function components are just regular JavaScript functions:

```
function Hello(props) {
    return <div>Hello {props.name}</div>;
}
```

To be classified as a valid React component, the function has to fulfill two criteria:

1. It has to either return an explicit null (undefined is not allowed) or a valid React.element (in the form of JSX).
2. It receives a props object.

Creating our first component

Let's create a simple functional component that displays a greeting message.

Greeting.js

```
function Greeting() {
  return <h1>Hello, World!</h1>;
}
export default Greeting;
```

- **Lines 1–3:** We define a function Greeting, which is our component.

- **Line 2:** The function returns an <h1> element with the text Hello, World!.
- **Line 4:** We export the Greeting component so it can be used in other parts of our application.

Rendering the component

To display this component on the screen, we'll render it within our main application file.

Click here to download the code for Main application file rendering the Greeting component

- **Line 1:** We import the Greeting component we just created.
- **Lines 3–9:** We define the App component, which is the root component of our application.
- **Lines 5–7:** Inside the App component's return statement, we include the <Greeting /> component within a .
- **Line 11:** We export the App component as the default export.

Exporting components makes them available for use in other parts of the application. It promotes modularity and better code organization. If we don't export App, other files won't know it exists, and we won't be able to use it elsewhere.

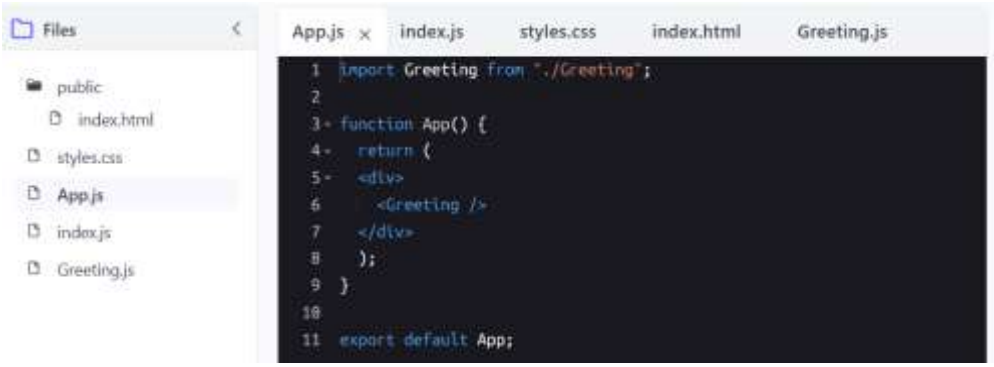
Components hierarchy

As we start working with components in React, let's first see how they can be organized and interact with each other. This organization is known as the **components hierarchy**.

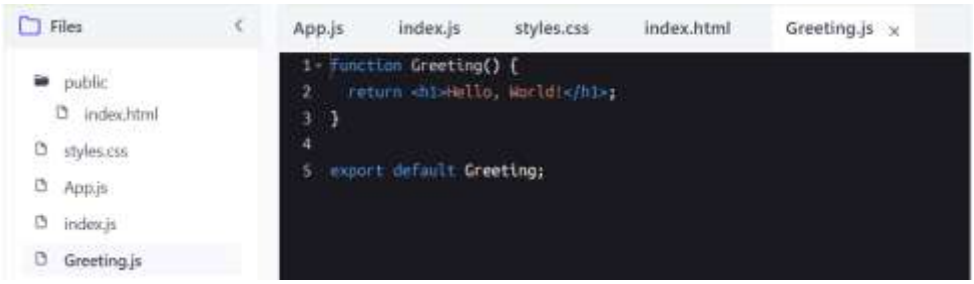
The components hierarchy represent the structure of the React application, showing how components are nested within one another. Think of it like a family tree, where each component can have parent and child relationships.

Simple example of components hierarchy

Let's revisit the App and Greeting components we created earlier:



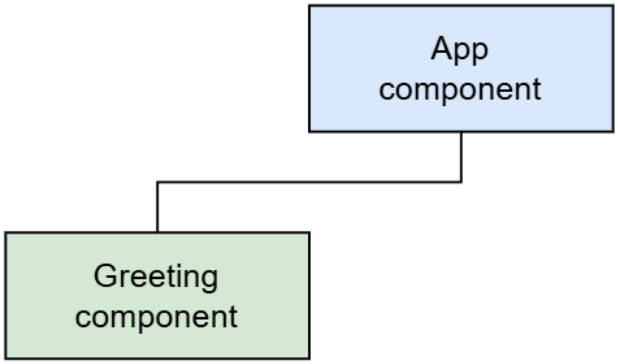
The App component rendering the Greeting component



The Greeting component that displays a greeting message

Understanding the relationship

- **Parent component:** App is the parent component.
- **Child component:** Greeting is the child component that App renders.



A simple components hierarchy with App as the parent and Greeting as the child

This shows that the App component contains the Greeting component.

Adding more components

Let's say we add another component called Footer:

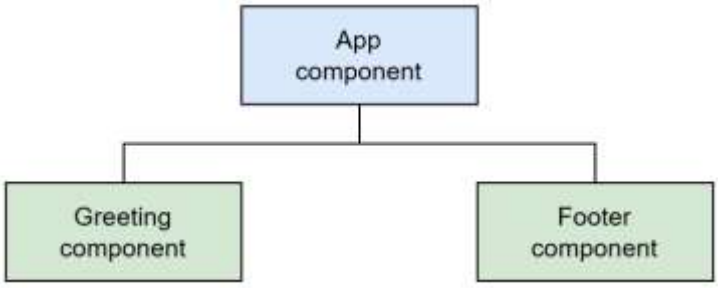
A Footer component that displays a footer message

We can update our App component to include Footer:

Click here to download the updated code along with Footer.js

Updated hierarchy

Now, the components hierarchy look like this:



A components hierarchy with App as the parent, and Greeting and Footer components as child

Passing Props to Components

Building dynamic and flexible React applications is impossible without passing data between components. This is where **props** come into play, allowing components to be reusable and interactive.

Understanding props in React

Props (short for "properties") are a mechanism for passing data from a parent component to a child component. They allow components to be dynamic and reusable by enabling them to receive input data and render accordingly.

Why props matter

- **Data flow:** Props facilitate the flow of data in a unidirectional (one-way) manner from parent to child components.
- **Reusability:** By accepting props, components become more flexible and can be reused with different data.
- **Customization:** Props allow us to customize components, making them adaptable to various situations.

Passing props to components

Let's start by creating a simple Message component that accepts props.

Message.js

```
1 function Message(props) {
2   return <p>{props.text}</p>;
3 }
4
5 export default Message;
```

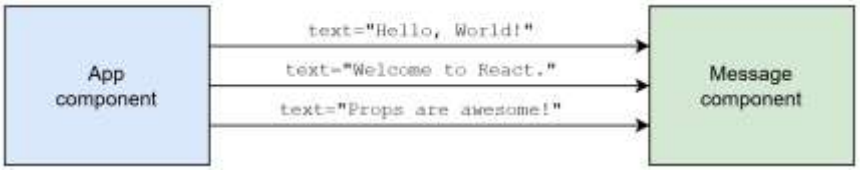
A Message component that displays a text passed via props

- **Lines 1–3:** We define the Message functional component that accepts props as an argument.
- **Line 2:** The component returns a <p> element displaying {props.text}, which accesses the text prop.

Now, let's use the Message component in our App component and pass different messages as props.

Click to download for App component passing different text props to the Message component

- **Line 1:** We import the Message component.
- **Lines 3–11:** We define the App component.
- **Lines5–9:** Inside the App component's return statement, we render the Message component three times, each with a different text prop.



The App component rendering the Message component three times with different text props

Understanding the props object

In the Message component, props is an object that contains all the props passed to it. We can access each prop using dot notation.

Accessing multiple props

Let's modify the Message component to accept a text and a color prop.

Message.js

```
1 function Message(props) {
2   return <p style={{ color: props.color }}>{props.text}</p>;
3 }
4
5 export default Message;
```

The Message component now accepts text and color props

- **Line 2:** We add an inline style to the <p> element, setting its color to props.color.

Passing multiple props

Update the App component to pass the new color prop.

Click here to download for Passing both text and color props to the Message component

- **Lines 6–8:** We pass both text and color props to each Message component.

Using props to make components dynamic

Components can render dynamic content by accepting props based on the data they receive. This is similar to how the color information was passed as a prop in the previous section.

Creating a UserCard component

Let's create a UserCard component that displays user information.

UserCard.js

```
1  function UserCard(props) {
2    return (
3    <div>
4      <h2>{props.name}</h2>
5      <p>Age: {props.age}</p>
6      <p>Email: {props.email}</p>
7    </div>
8    );
9  }
10
11 export default UserCard;
```

A UserCard component displaying user details passed via props

- **Lines 3–7:** The component returns a containing:
- **Line 4:** An <h2> element displaying the user's name.
- **Line 5:** A <p> element displaying the user's age.
- **Line 6:** A <p> element displaying the user's email.

Rendering the UserCard component

Let's render the UserCard component.

Click here to download the code for Rendering UserCard components with different user data

Styling Components in React

Styling improves user interface and user experience both. It brings life to our components, making our application visually appealing and user-friendly. React supports various methods of styling, allowing us to choose the one that best fits our project's needs.

Different ways to style components

In React, there are three ways to apply styles to our components.

Using CSS stylesheets

This is the traditional way of styling web pages, and it works seamlessly with React. Let's create a CSS file named styles.css in our project directory.

A CSS file defining styles for .header and .paragraph classes, and body and h1 elements

We can apply these styles using the className attribute. In JSX, className is used instead of class to avoid conflicts with the JavaScript class keyword.

Click here to download the code for Applying CSS classes to elements using className

In index.js:

- **Line 3:** We import the styles.css file so that the styles are applied globally. When we import the CSS file in our component, the styles become available to that component and any child components.

In App.js:

- **Line 4:** The h1 element uses className="header" to apply the .header class styles.
- **Line 5:** The p element uses className="paragraph" to apply the .paragraph class styles.

Inline styles

Inline styles are applied directly to the elements using the style attribute, which accepts a JavaScript object.

Click here to download the code for Applying inline styles using the style attribute

- **Lines 2–7:** We define a headerStyle object containing CSS properties in camelCase instead of kebab-case.
- **Line 11:** We apply headerStyle to the h1 element using style={headerStyle}.
- **Line 12:** We apply inline styles directly within the style attribute using double curly braces ({{ }}).

Inline styles can use variables and expressions, making them suitable for dynamic styling.

CSS modules

CSS modules allow us to write CSS in separate files but scope the styles locally to the component, preventing class name collisions. CSS modules automatically generate unique class names at build time, ensuring that styles are scoped locally to the component. This means we can use the same class names in different components without worrying about class name collisions or unintended style overrides, promoting better modularity and maintainability in our application.

Let's create a CSS file with the .module.css extension:

A CSS module file for the Header component

To apply the styles from the CSS module to our component, import the module and use the class names via the className attribute.

Click here to download the code for Applying styles from a CSS module using className

- **Line 1:** We import the styles as an object named styles.
- **Line 4:** We apply the class using className={styles.header}.
- **Locally scoped:** The class names are scoped locally to the component, preventing conflicts.
- **Unique class names:** Under the hood, CSS modules generate unique class names.

Comparison of styling methods in React

Let's have a brief comparison of all the three ways to style components that we discussed: using CSS stylesheets, inline styles, and CSS modules.

Aspect	CSS stylesheets	Inline styles	CSS modules
Definition	External `.css` files applied globally	Styles applied directly to elements via the `style` attribute	`.module.css` files where class names are scoped locally
Usage syntax	`className="className"`	`style={{ key: 'value' }}`	`className={styles.className}`
Scope	Global	Inline (element-specific)	Local to the component
Reusability	High (styles can be reused across components)	Low (styles are tied to specific elements)	Moderate (styles are component-specific)
Dynamic styling	Less straightforward	Highly dynamic using JavaScript expressions	Supports dynamic styling but requires additional setup
CSS features supported	All standard CSS	Limited (no pseudo-classes or media queries)	All standard CSS
Class name collisions	Possible (need to manage unique class names)	Not applicable	Avoided due to locally scoped class names
Maintenance	Can become complex in large projects	Can lead to cluttered JSX code	Easier maintenance with modular files
Styling complexity	Good for complex stylesheets	Suitable for simple, component-specific styles	Good for complex styles within a component
Performance	Efficient (styles are cached by the browser)	Less efficient (styles are applied inline per element)	Efficient (similar to CSS Stylesheets)
Best use case	Best for global styles shared across multiple components	Best for dynamic, component-specific styles that depend on state or props	Best for scoped styles within a component to avoid conflicts
Example usage	`<div className="container">`	`<div style={{ margin: '10px' }}>`	`<div className={styles.container}>`

Handling Events in React Components

Interactivity is a core aspect of modern web applications, and React provides a straightforward way to handle user interactions and events. React handles events similarly to how events are handled in regular HTML and JavaScript, but with some syntactical differences. Let's see how we can handle events in React.

Why event handling matters

- **Interactivity:** Events allow users to interact with our application.
- **Dynamic behavior:** Event handling enables components to respond to user input, such as clicks, typing, and form submissions.
- **User experience:** Proper event handling enhances the user experience by making applications more intuitive and responsive.

Understanding event handling in React

In React, events are named using camelCase, rather than lowercase. For example, the HTML onclick event is written as onClick in React.

Comparing HTML and React event handling syntax

Key differences

- **Event naming:** Use camelCase (e.g., onClick, onChange) in React.
- **Event handling:** Pass a function reference as the event handler, not a string.

Creating event handler functions

Event handler functions define what happens when an event occurs. In React, these are typically defined within the component. Let's create a simple button that displays an alert when clicked.

Click here to download the code for A button component with an onClick event handler

Common events in React

- onClick**:** Triggered when an element is clicked.
- onChange**:** Triggered when the value of an input element changes.
- onSubmit**:** Triggered when a form is submitted.
- onMouseOver**:** Triggered when the mouse pointer moves over an element.
- onFocus / onBlur**:** Triggered when an element gains or loses focus.

Passing arguments to event handlers

We might need to pass arguments to our event handlers. We can do this by wrapping the event handler in an arrow function.

Click here to download code for Passing arguments to an event handler using an arrow function

- **Line 2:** The handleClick function accepts a message parameter.
- **Line 8:** We pass an anonymous arrow function to onClick that calls handleClick with a specific message.

Handling form events

Forms are integral to web applications. React simplifies handling form inputs and submissions. Let's create a component that logs the input value to the console whenever it changes.

Click here to download the code for An input field that logs the input value on change

- **Lines 2–4:** The handleChange function logs the current value of the input field to the console.
- **Line 10:** We attach the handleChange function to the input's onChange event.

Preventing default behavior

Sometimes, we may need to prevent the default action of an event, such as preventing a form from submitting and refreshing the page.

Click to download the code for A form that handles submission without refreshing the page

- **Lines 2–5:** The handleSubmit function handles the form submission event.
- **Line 3:** event.preventDefault() prevents the default form submission behavior (refreshing the page).
- **Line 8:** We attach the handleSubmit function to the form's onSubmit event.

Passing event handlers as props

We can pass event handlers to child components via props, promoting modularity and reusability.

Parent component

In the parent component, we'll define an event handler and pass it to the child component via props.

- **Line 1:** We import the Button component.
- **Lines 4–6:** We define the showMessage function.
- **Line 10:** We pass showMessage as the onClick prop and set the label prop.

The App component passing an onClick handler to the Button component

Child component

In the child component, we'll receive the event handler via props and attach it to an element's event.

The Button component receiving onClick and label props

- **Line 3:** We attach props.onClick to the button's onClick event.
- **Line 4:** We display the button's label using props.label.

Click here to download the code

Creating Stateless Components

In large React application, ensuring that our components are both flexible and easy to maintain is of supreme importance. By creating **stateless components**, we can build pieces of our UI that rely entirely on their inputs (props) and do not manage any internal state. This approach simplifies our code and makes our components highly reusable, allowing us to integrate them in various parts of our application with minimal adjustments.

Stateless reusable components

Stateless reusable components are functional components that do not hold any internal state. Instead, they receive all the information they need through props. By doing so, these components become more predictable, easier to test, and more flexible in different contexts.

Before we dive into the code, consider these guiding principles for creating a stateless reusable component:

- **Single responsibility:** Each component should do one thing well, like displaying a label or rendering a list of items.
- **Props-driven:** All dynamic data, event handlers, and styling options should come through props.
- **No internal state:** Avoid holding or mutating any internal state.

Creating a stateless Button component

Let's create a Button component that only needs a label (the text displayed) and an onClick handler (a function passed as a prop) to work. By not managing its own state, this

button is easy to plug into various scenarios—such as saving data, canceling an action, or navigating somewhere—just by changing its props.

The following code shows a Button component that relies entirely on props to determine its appearance and behavior.

Button.js

```
1  function Button(props) {
2    return (
3    <button onClick={props.onClick} style={props.style}>
4      {props.label}
5    </button>
6    );
7  }
8
9  export default Button;
```

The Button component uses props for its label, styling, and click behavior

- **Line 3:** We attach the onClick handler passed via props.onClick. This allows the parent component to define what happens when the button is clicked. The style prop provides flexibility in visual customization. Different parts of our app can style the same Button component differently without modifying the component's code.
- **Line 4:** The button's visible text is determined by props.label, making it easy to change the button's label based on where it's used.

By keeping the component stateless, we ensure that its behavior and appearance depend solely on the data and callbacks passed to it. This makes Button a reusable building block that we can integrate into various parts of the UI.

Integrating Button into an App component

Once we have a stateless component like Button, we can use it anywhere by simply passing the appropriate props. For example, let's integrate the Button component into an App component, passing different labels and event handlers.

The following code demonstrates how to reuse the Button component with different props in a parent App component.

Click here App component reusing the Button component with different props and event handlers

- **Lines 4–10:** We define handleSave and handleCancel functions that show different alerts. Passing these functions as props to our Button component tailors its behavior for each scenario.
- **Lines 14–24:** We render the Button component twice with different labels, click handlers, and styles, demonstrating the component's flexibility.

Hooks: Managing States and Effects in React Components

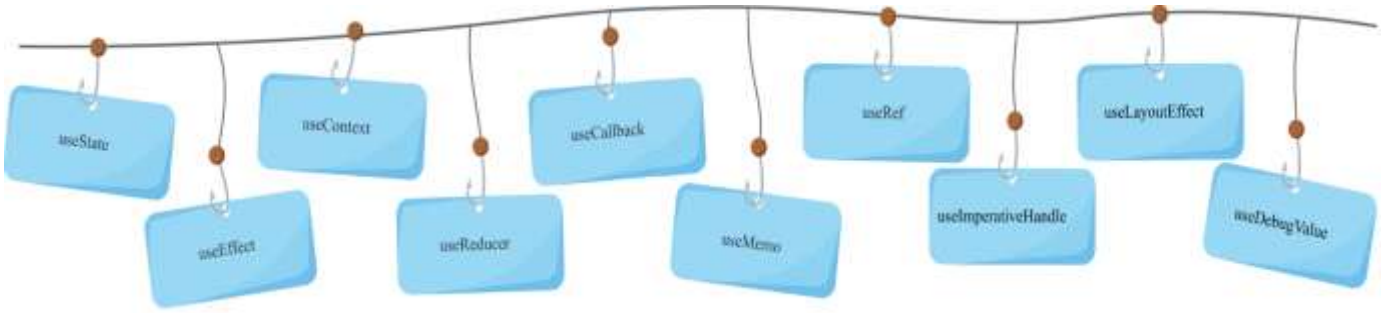
Understanding Hooks in React

In modern React development, **Hooks** were introduced to simplify code structure and enhance the capabilities of functional components. Previously, developers relied heavily on class components to handle internal state and side effects. With Hooks, it's now possible to manage

these features directly within simple functions, making code more readable, maintainable, and intuitive. Understanding Hooks is essential for building scalable, reactive interfaces that adapt seamlessly to user interactions and data changes.

What are Hooks, exactly?

React Hooks are special functions that let us use state and other React features without writing class components. The primary goal is to simplify how we manage data (state) and tasks, such as fetching data, updating the browser title, or running timers (effects), within UI components. By using Hooks, our functional components can become more powerful while remaining concise and easy to read.



Key Hooks in React

Let’s have a look at a basic greeting functional component:

```
index.js
1 function Greeting() {
2   return (
3     <h1>Hello, welcome to our React application!</h1>
4     );
5 }
6
7 export default Greeting;
```

A basic functional component before using Hooks

At this point, the Greeting component is entirely static and stateless. It can display text, but it cannot hold or manage any internal data that changes over time.

Before Hooks, functional components were limited to just rendering UIs.

How Hooks solve the statelessness problem

To appreciate the power of Hooks, consider how we can enhance a functional component by giving it the ability to remember values and respond to user interactions. Even though we’re not diving deep into the details right now, this quick preview shows how a formerly “stateless” functional component can manage its own data and react to changes without being converted into a class. Let's have a look at the above greeting functional component after managing its state using Hooks.

Click here to download the code for Enhancing greeting functional component with Hooks

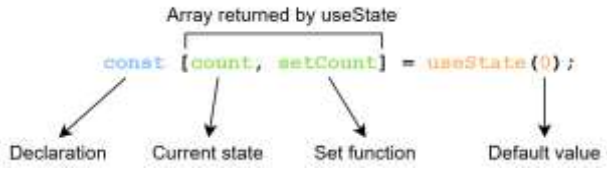
By adding useState, the component now maintains its own data and reacts to user actions, transforming from a static, “read-only” UI into a dynamic, “interactive” one. All this is done without using class components.

The useState Hook

In building dynamic, user-friendly interfaces, it's essential to let components remember and update their own data. Previously, only class components could hold internal state, making certain interactions more complex to implement. Now, with the useState Hook, we can manage state directly inside the functional components. This addition enables our UI to respond immediately to user inputs, changing data, and other events, with simpler, more readable code.

How useState works

The useState hook allows functional components to have their own internal state, enabling them to manage dynamic data over time. When we call useState, we provide a default value, which sets the initial state of the component. It returns an array containing two elements: the current state value and a function to update that state. These are typically destructured into variables, like [count, setCount].

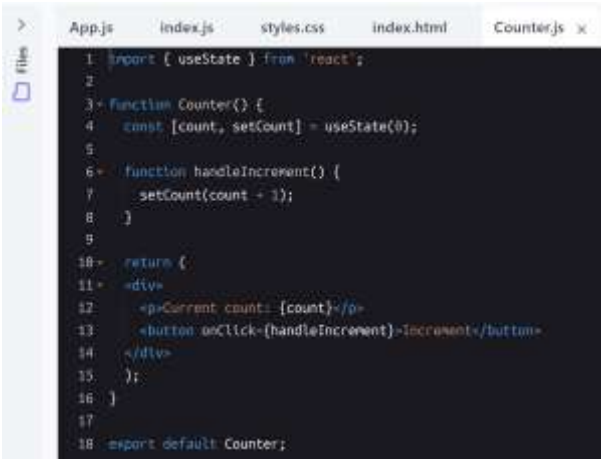


The first variable (count) holds the current value of the state, while the second variable (setCount) is a function used to update the state. When the updater function is called with a new value, React automatically re-renders the component, ensuring the UI reflects the updated state. This process makes it easy to handle dynamic, interactive behavior in React components, such as tracking user interactions or updating content based on events.

Adding state to functional components

Let's have a look at a component that uses useState to keep track of a counter:

- **Line 4:** useState(0) initializes a state variable count with a starting value of 0, and provides a setCount function to update it.
- **Lines 7:** Whenever setCount is called, React re-renders the component, showing the updated count in **line 12**.
- **Line 13:** The button triggers the increment function (handleIncrement), demonstrating how easily state management can turn a static component into a dynamic, interactive experience.



A Counter component using useState

We can also pass an arrow function to setCount(). This is particularly useful when the new state depends on the previous state. The arrow function we pass receives the current state as an argument and returns the updated state.

Click to download the code for A counter component using an arrow function with setCount

- **Line 7:** The arrow function (setCount((prevCount) => prevCount + 1)) ensures that the correct value is used by referencing the latest state and avoiding race conditions.
    - prevCount**:** The current state value (provided automatically by React).

The useEffect Hook

In any application, certain actions or "side effects" need to occur outside the regular rendering cycle. In the context of React and programming in general, **side effects** refer to operations or actions performed by a function that interact with or modify something outside the function's scope, or affect the application in ways that are not purely computational. Examples include fetching data from an API, updating the DOM, or setting up timers.

In React, the useEffect Hook allows us to handle these side effects in functional components without relying on the component's life cycle methods. This makes managing external interactions and cleanup tasks much more intuitive.

Understanding useEffect

The useEffect Hook is used for performing side effects in our components. It runs after the component renders and can optionally re-run when specific values (called dependencies) change.

What are dependencies?

Dependencies are the values or state variables that, when changed, trigger the re-execution of the effect function provided to useEffect. These values are listed in an array as the second parameter of useEffect.

useEffect(<function>, <dependency>)

```
import { useState, useEffect } from 'react';
import { createRoot } from 'react-dom/client';

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });

  return <h1>I've rendered {count} times!</h1>;
}

createRoot(document.getElementById('root')).render(
  <Timer />
);
```

The useContext Hook

React Context

React Context is a way to manage state globally.

It can be used together with the useState Hook to share state between deeply nested components more easily than with useState alone.

The Problem

State should be held by the highest parent component in the stack that requires access to the state.

To illustrate, we have many nested components. The component at the top and bottom of the stack need access to the state.

To do this without Context, we will need to pass the state as "props" through each nested component. This is called "prop drilling".

Example:

Passing "props" through nested components:

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

function Component1() {
  const [user, setUser] = useState("Linus");
```

```
  return (
    <>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 user={user} />
    </>
  );
}

function Component2({ user }) {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 user={user} />
    </>
  );
}

function Component3({ user }) {
  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <Component1 />
);
```

Even though component 2 did not need the state, it had to pass the state along so that it could reach component 3.

The Solution

The solution is to create context.

Create Context

To create context, you must Import createContext and initialize it:

```
import { useState, createContext, useContext } from 'react';
import { createRoot } from 'react-dom/client';

const UserContext = createContext();
```

Next we'll use the Context Provider to wrap the tree of components that need the state Context.

Context Provider

Wrap child components in the Context Provider and supply the state value.

```
function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    <UserContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 />
    </UserContext.Provider>
  );
}
```

Now, all components in this tree will have access to the user Context.

Use the useContext Hook

In order to use the Context in a child component, we need to access it using the useContext Hook.

First, include the useContext in the import statement:

```
import { useState, createContext, useContext } from "react";
```

Then you can access the user Context in all components:

```
function Component3() {
  const user = useContext(UserContext);

  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
```

Full Example

Example:

Here is the full example using React Context:

```
import { useState, createContext, useContext } from 'react';
import { createRoot } from 'react-dom/client';

const UserContext = createContext();

function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    <UserContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 />
    </UserContext.Provider>
  );
```

```
}

function Component2() {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 />
    </>
  );
}

function Component3() {
  const user = useContext(UserContext);

  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <Component1 />
);
```

The useRef Hook

The useRef Hook allows you to persist values between renders.

It can be used to store a mutable value that does not cause a re-render when updated.

It can be used to access a DOM element directly.

Does Not Cause Re-renders

If we tried to count how many times our application renders using the useState Hook, we would be caught in an infinite loop since this Hook itself causes a re-render.

To avoid this, we can use the useRef Hook.

Example:

Use useRef to track application renders.

```
import { useState, useRef, useEffect } from 'react';

import { createRoot } from 'react-dom/client';
```

```
function App() {
  const [inputValue, setInputValue] = useState("");
  const count = useRef(0);

  useEffect(() => {
    count.current = count.current + 1;
  });

  return (
    <>
      <p>Type in the input field:</p>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h1>Render Count: {count.current}</h1>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <App />
);
```

useRef() only returns one item. It returns an Object called current.

When we initialize useRef we set the initial value: useRef(0).

It's like doing this: const count = {current: 0}. We can access the count by using count.current.

The useMemo Hook

The React useMemo Hook returns a memoized value.

Think of memoization as caching a value so that it does not need to be recalculated.

The useMemo Hook only runs when one of its dependencies update.

This can improve performance.

The useMemo and useCallback Hooks are similar:

useMemo returns a memoized value.

useCallback returns a memoized function.

Learn more about useCallback in the useCallback chapter.

Without useMemo

The useMemo Hook can be used to keep expensive, resource intensive functions from needlessly running.

In this example, we have an expensive function that runs on every render.

When changing the count or adding a todo, you will notice a delay in execution.

Example:

A poor performing function. The expensiveCalculation function runs on every render:

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

const App = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState([]);
  const calculation = expensiveCalculation(count);

  const increment = () => {
    setCount((c) => c + 1);
  };
  const addTodo = () => {
    setTodos((t) => [...t, "New Todo"]);
```

```
  };

  return (
    <div>
      <div>
        <h2>My Todos</h2>
        {todos.map((todo, index) => {
          return <p key={index}>{todo}</p>;
        })}
        <button onClick={addTodo}>Add Todo</button>
      </div>
      <hr />
      <div>
        Count: {count}
        <button onClick={increment}>+</button>
        <h2>Expensive Calculation</h2>
        {calculation}
        <p>Note that this example executes the expensive function also when you click on the Add Todo button.</p>
      </div>
    </div>
  );
};

const expensiveCalculation = (num) => {
  console.log("Calculating...");
  for (let i = 0; i < 1000000000; i++) {
    num += 1;
  }
  return num;
};
```

```
createRoot(document.getElementById('root')).render(
  <App />
);
```

Use useMemo

To fix this performance issue, we can use the useMemo Hook to memoize the expensiveCalculation function. This will cause the function to only run when needed.

We can wrap the expensive function call with useMemo.

The useMemoHook accepts a second parameter to declare dependencies. The expensive function will only run when its dependencies have changed.

In the following example, the expensive function will only run when count is changed and not when todo's are added.

Example:

Performance example using the useMemo Hook:

```
import { useState, useMemo } from 'react';
import { createRoot } from 'react-dom/client';

const App = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState([]);
  const calculation = useMemo(() => expensiveCalculation(count), [count]);

  const increment = () => {
    setCount((c) => c + 1);
  };
  const addTodo = () => {
    setTodos((t) => [...t, "New Todo"]);
  };

  return (
    <div>
```

```
      <div>
        <h2>My Todos</h2>
        {todos.map((todo, index) => {
          return <p key={index}>{todo}</p>;
        })}
        <button onClick={addTodo}>Add Todo</button>
      </div>
      <hr />
      <div>
        Count: {count}
        <button onClick={increment}>+</button>
        <h2>Expensive Calculation</h2>
        {calculation}
      </div>
    </div>
  );
};

const expensiveCalculation = (num) => {
  console.log("Calculating...");
  for (let i = 0; i < 1000000000; i++) {
    num += 1;
  }
  return num;
};

createRoot(document.getElementById('root')).render(
  <App />
);
```

What is React Router?

React Router is a library that provides routing capabilities for React applications.

Routing means handling navigation between different views.

React Router is the standard routing library for React applications. It enables you to:

- Create multiple pages in your single-page application
- Handle URL parameters and query strings
- Manage browser history and navigation
- Create nested routes and layouts
- Implement protected routes for authentication

Without a router, your React application would be limited to a single page with no way to navigate between different views.

Install React Router

In the command line, navigate to your project directory and run the following command to install the package:

npm install react-router-dom

Wrap Your App with BrowserRouter

Your application must be wrapped with the BrowserRouter component to enable routing:

```
function App() {
  return (
    <BrowserRouter>
      {/* Your app content */}
    </BrowserRouter>
  );
}
```

Create Views

To demonstrate routing, we'll create three pages (or views) in our application: Home, About, and Contact:

We will create all three views in the same file for simplicity, but you can of course split them into separate files.

Example

```
function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function Contact() {
  return <h1>Contact Page</h1>;
}
```

Basic Routing

React Router uses three main components for basic routing:

- Link: Creates navigation links that update the URL
- Routes: A container for all your route definitions
- Route: Defines a mapping between a URL path and a component

Let's add navigation links and routes for each link:

Example

Note that we need to import BrowserRouter, Routes, Route, Link from 'react-router-dom'.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
```

```
}

function Contact() {
  return <h1>Contact Page</h1>;
}

function App() {
  return (
    <BrowserRouter>
      {/* Navigation */}
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>

      {/* Routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

In this example:

- BrowserRouter wraps your app and enables routing functionality
- Link components create navigation links
- Routes and Route define your routing configuration

Nested Routes

You can have a Route inside another Route, this is called nested routes.

Nested routes allow you change parts of the page when you navigate to a new URL, while other parts is not changed or reloaded, almost like having a page within a page.

Let's use the example above, and add two new components that will be rendered inside the Products component.

One called CarProducts and one called BikeProducts:

Example

Note that we also need to import the Outlet component from 'react-router-dom'.

```
import { BrowserRouter, Routes, Route, Link, Outlet } from 'react-router-dom';


function Home() {
  return <h1>Home Page</h1>;
}


function Products() {
  return (
    <div>
      <h1>Products Page</h1>
      <nav style={{ marginBottom: '20px' }}>
        <Link to="/products/car">Cars</Link> |{" "}
        <Link to="/products/bike">Bikes</Link>
      </nav>
      <Outlet />
    </div>
  );
}


function CarProducts() {
  return (
```

```
    <div>
      <h2>Cars</h2>
      <ul>
        <li>Audi</li>
        <li>BMW</li>
        <li>Volvo</li>
      </ul>
    </div>
  );
}

function BikeProducts() {
  return (
    <div>
      <h2>Bikes</h2>
      <ul>
        <li>Yamaha</li>
        <li>Suzuki</li>
        <li>Honda</li>
      </ul>
    </div>
  );
}

function Contact() {
  return <h1>Contact Page</h1>;
}

function App() {
  return (
```

```
    <BrowserRouter>
      {/* Navigation */}
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/products">Products</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>

      {/* Routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />}>
          <Route path="car" element={<CarProducts />} />
          <Route path="bike" element={<BikeProducts />} />
        </Route>
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Important notes about the example above:

1. **Outlet:**
   The <Outlet /> element in the Products component specifies where to render the child route's content.
2. **Routes:**
   The Routes element contains the routes to CarProducts and BikeProducts as child routes of the Products parent route.
3. **URL Structure:**
   The URL structure is relative to the parent route's path. For example:
   - When you navigate to '/products/car', the CarProducts component is rendered.
   - When you navigate to '/products/bike', the BikeProducts component is rendered.